

SIMATS ENGINEERING

ASSESSMENT TOOL 1

NAME: HARITHATCHANI R

REG NO: 192421330

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0703

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards

(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

Aim

To develop a Python program that simulates a Star Topology in which multiple hosts are connected to a central Ethernet switch using Cat6 UTP cables.

Components

- PCs/Hosts
- Ethernet Switch
- Cat6 UTP cables
- RJ-45 connectors
- Python for simulation

Algorithm

1. Start the program.
2. Read the number of hosts.
3. Create a central switch.
4. Connect every host to the central switch.
5. Assign a Cat6 cable length to each connection.
6. Display the host, switch, and cable length.
7. Verify that every host has a connection to the switch.
8. Stop.

Python Program

```
# Star Topology Simulation

def star_topology():

    try:

        n = int(input("Enter number of hosts: "))

        if n <= 0:

            print("Number of hosts must be greater than zero.")

            return

        switch = "Central Switch"

        print("\n===== STAR TOPOLOGY =====")

        print("Central Device:", switch)

        print("Transmission Medium: Cat6 UTP")

        print("-----")

        for i in range(1, n + 1):

            length = 10 + (i * 2)

            print(

                f"Host- {i} ---- {switch} "

                f"---- Cable Length: {length} m"

            )

        print("-----")

        print("All hosts are connected to the central switch.")

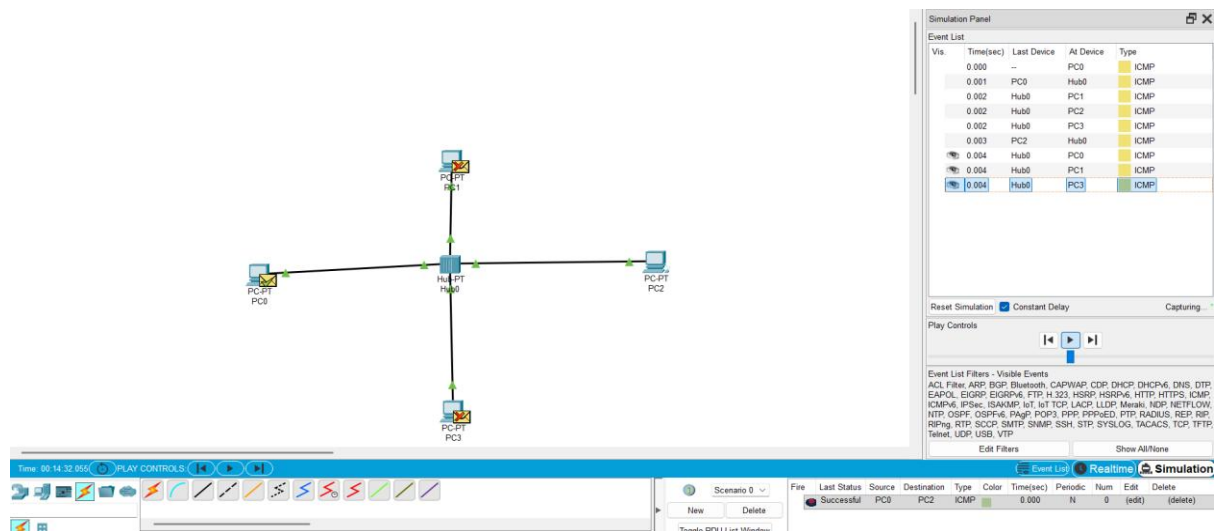
        print("=====")

    except ValueError:

        print("Invalid input.")

star_topology()
```

Sample Output



Result

The star topology was successfully simulated. All hosts were connected independently to the central Ethernet switch using Cat6 UTP cables.

Networking Significance

Star topology is widely used in modern LANs because:

- Each host has a dedicated connection.
- Failure of one cable normally affects only one host.
- Troubleshooting is easier.
- A switch can intelligently forward frames.
- Additional hosts can easily be connected.

2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

Aim

To develop a Python function `validate_segment(length_m)` that verifies whether a UTP Ethernet cable segment is within the commonly used 100-meter maximum Ethernet channel/segment limit.

Algorithm

1. Start.
2. Accept cable length in meters.
3. Check whether the length is negative.
4. If $\text{length} \leq 100$ m, consider it valid.
5. If $\text{length} > 100$ m, display an error.
6. Test the function using three different values.
7. Display the results.
8. Stop.

Python Program

UTP Cable Length Validation

```
def validate_segment(length_m):
```

```
    if length_m < 0:
```

```
        return False, "Error: Cable length cannot be negative."
```

```
    if length_m <= 100:
```

```
        return True, "Valid UTP segment."
```

```
    return False, "Error: Cable length exceeds 100 meters."
```

Three test cases

```
test_cases = [50, 100, 120]
```

```
print("==== UTP CABLE VALIDATION =====")
```

```
for length in test_cases:
```

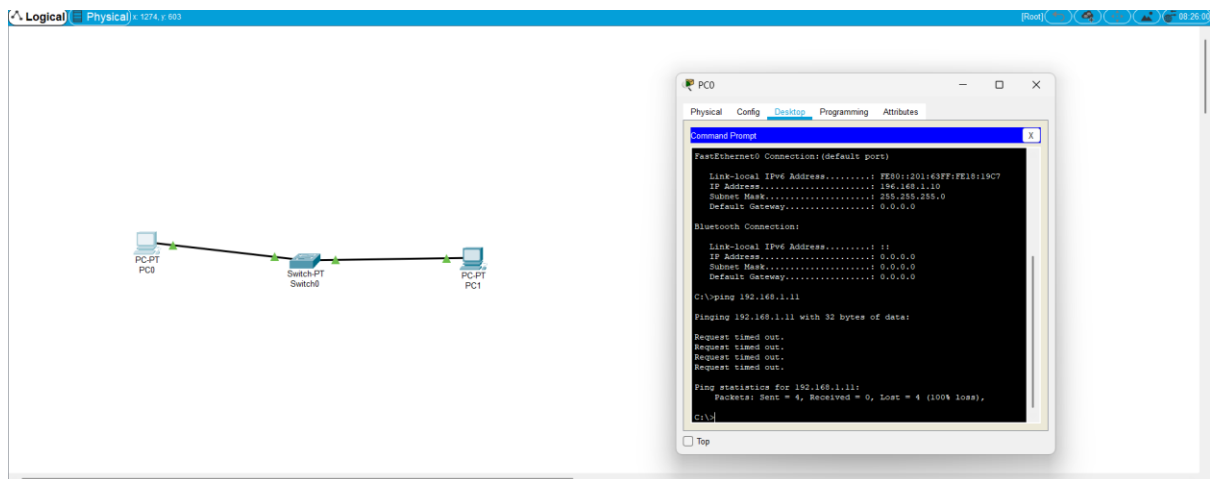
```
    result, message = validate_segment(length)
```

```
    print("\nCable Length:", length, "meters")
```

```
    print(message)
```

```
    print("Result:", result)
```

Sample Output



Result

The Python program successfully validated UTP cable lengths and identified cable segments exceeding the specified 100-meter limit.

Networking Significance

Maintaining appropriate cable lengths helps ensure:

- Reliable signal transmission
- Reduced attenuation
- Stable Ethernet communication
- Proper LAN performance

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

Aim

To simulate the operation of Carrier Sense Multiple Access with Collision Detection (CSMA/CD) for four hosts and demonstrate collision detection, random backoff, and retransmission.

Algorithm

1. Create four hosts.
2. Select a host for transmission.
3. Sense the shared medium.
4. Attempt transmission.
5. Randomly simulate whether a collision occurs.
6. If collision occurs, record it.
7. Generate a random backoff interval.
8. Wait for the backoff period.
9. Retransmit.
10. Record all events in csma_log.txt.

Python Program

```
# CSMA/CD Simulation
```

```
import random
```

```
import time
```

```
LOG_FILE = "csma_log.txt"
```

```
hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]
```

```
def log_event(message):
```

```
    print(message)
```

```
    with open(LOG_FILE, "a") as file:
```

```
        file.write(message + "\n")
```

```
# Create/clear log file
```

```
with open(LOG_FILE, "w") as file:
```

```
    file.write("==== CSMA/CD LOG =====\n")
```

```
def transmit(host):
```

```
    max_attempts = 5
```

```
    for attempt in range(1, max_attempts + 1):
```

```
        log_event(f'{host}: Transmission attempt {attempt}')
```

```

collision = random.choice([True, False, False])

if collision:

    log_event(f" {host}: Collision detected" )

    backoff = random.randint(1, 5)

    log_event(f" {host}: Backoff = {backoff} time units" )

    time.sleep(0.2)

else:

    log_event(f" {host}: Frame transmitted successfully" )

return

log_event( f" {host}: Transmission failed" )

print("===== CSMA/CD SIMULATION =====")

for host in hosts:

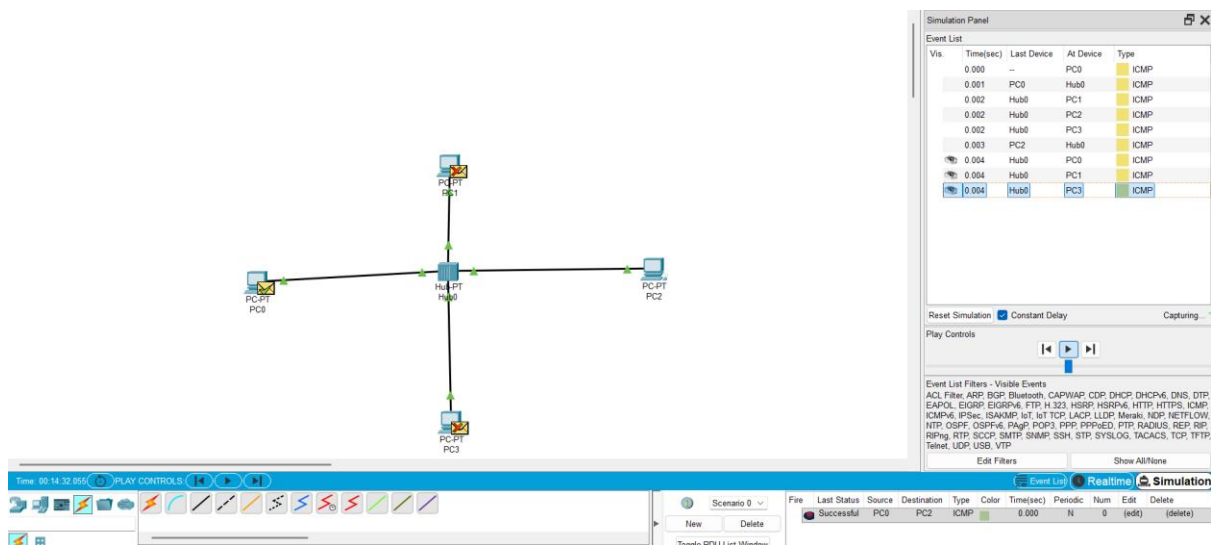
    transmit(host)

print("\nSimulation completed.")

print("Log file:", LOG_FILE)

```

Sample Output



Networking Significance

CSMA/CD reduces repeated transmission conflicts by:

- Detecting collisions.
- Stopping the affected transmission.
- Introducing a random waiting period.
- Retransmitting after the backoff period.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

Aim

To develop a Python client-server application that simulates communication between two hosts connected through a central switch and demonstrates MAC address learning and frame forwarding.

Server Program

Save as server.py.

```
import socket

HOST = "127.0.0.1"

PORT = 5000

mac_table = {}

def display_mac_table():

    print("\n===== MAC ADDRESS TABLE =====")

    for mac, port in mac_table.items():

        print(f"{mac} -> {port}")

    print("=====")

server = socket.socket(
```

```

    socket.AF_INET,
    socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print("Central Switch Server")
print("Waiting for client...")
connection, address = server.accept()
print("Client connected:", address)
client_mac = "AA:BB:CC:DD:EE:01"
server_mac = "AA:BB:CC:DD:EE:02"
mac_table[client_mac] = "Port-1"
display_mac_table()
while True:
    data = connection.recv(1024)
    if not data:
        break
message = data.decode()
print("\nReceived Frame:", message)
mac_table[server_mac] = "Port-2"
print("Frame forwarded to destination.")
connection.send( "Frame received successfully.".encode())
display_mac_table()
connection.close()
server.close()

```

Client Program

Save as client.py.

```

import socket

HOST = "127.0.0.1"

```

```
PORT = 5000

client = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

client.connect((HOST, PORT))

print("Connected to central switch.")

while True:
    message = input("Enter frame data (exit to stop): ")
    if message.lower() == "exit":
        break
    client.send(message.encode())
    response = client.recv(1024)
    print("Server:", response.decode())

client.close()

print("Client disconnected.")
```

Execution

Terminal 1:

```
python3 server.py
```

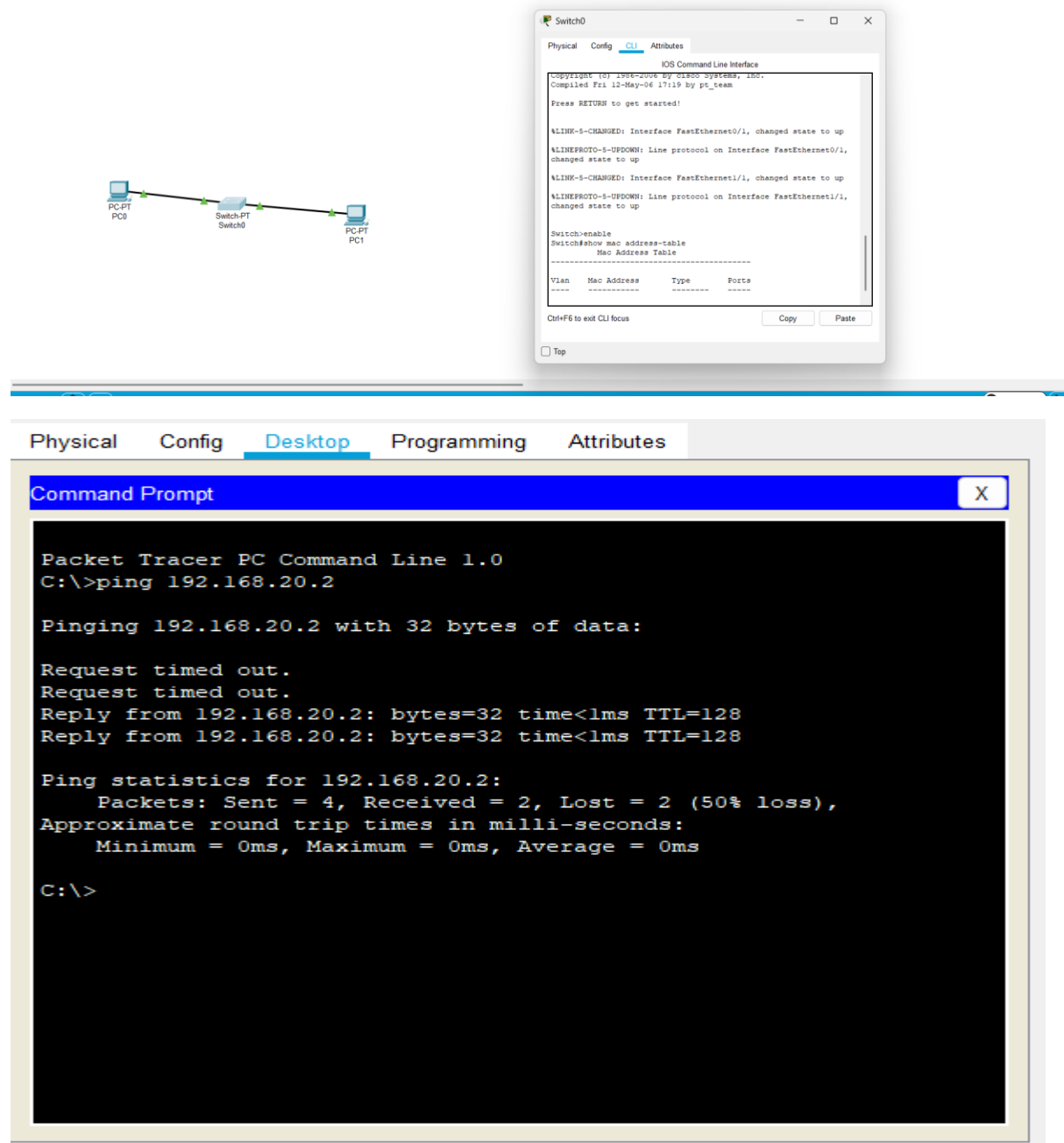
Terminal 2:

```
python3 client.py
```

Enter:

```
Hello Ethernet
```

Sample Output



Result

The client-server application successfully simulated Ethernet frame transmission and MAC address learning through a central switch.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

Aim

To design a Python class named StarSwitch that simulates the basic functions of an Ethernet switch, including adding ports, removing ports, and broadcasting frames.

Algorithm

1. Create the switch.
2. Add hosts to switch ports.
3. Maintain a list of active ports.
4. Receive a frame from a source host.
5. Identify all other active hosts.
6. Broadcast the frame to those hosts.
7. Record the source and destination hosts.
8. Remove a host when required.
9. Display the active ports.

Python Program

Ethernet Switch Simulation

```
class StarSwitch:
```

```
    def __init__(self):
        self.active_ports = []
        self.broadcast_log = []

    def add_port(self, host_id):
        if host_id not in self.active_ports:
            self.active_ports.append(host_id)
            print(host_id, "connected.")
        else:
```

```

        print(host_id, "already connected.")
def remove_port(self, host_id):
    if host_id in self.active_ports:
        self.active_ports.remove(host_id)
        print(host_id, "disconnected.")
    else:
        print(host_id, "not found.")
def broadcast(self, frame, source):
    if source not in self.active_ports:
        print("Source host is not connected.")
        return
    receivers = []
    for host in self.active_ports:
        if host != source:
            receivers.append(host)
    print("\n===== BROADCAST =====")
    print("Source:", source)
    print("Frame:", frame)
    for host in receivers:
        print("Frame received by:", host)
    self.broadcast_log.append(
        (source, receivers, frame) )
switch = StarSwitch()
switch.add_port("Host-1")
switch.add_port("Host-2")
switch.add_port("Host-3")
switch.add_port("Host-4")
print("\nActive Ports:")
print(switch.active_ports)

```

```

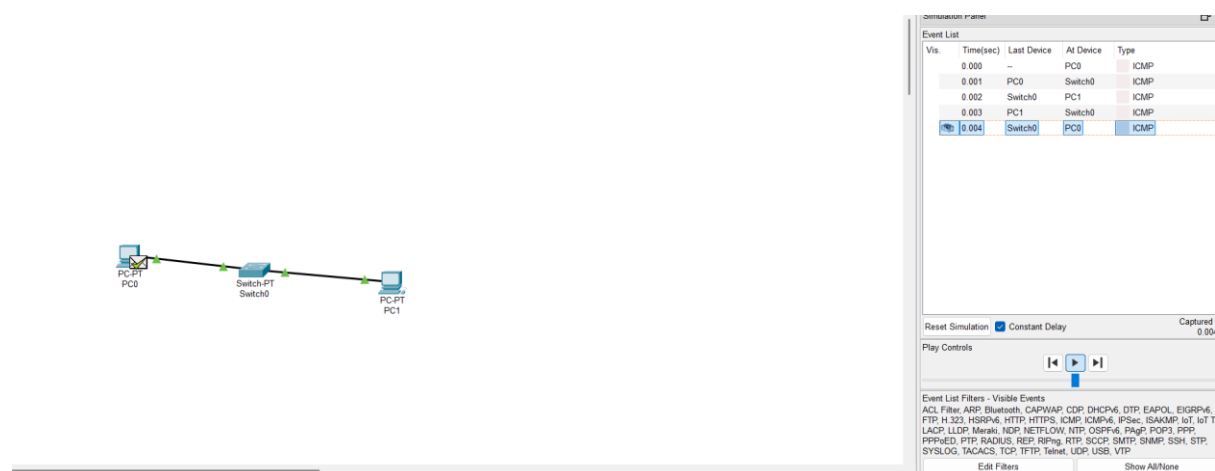
switch.broadcast(
    "Ethernet Frame 001",
    "Host-1"
)

switch.broadcast(
    "Ethernet Frame 002",
    "Host-3"
)

switch.remove_port("Host-4")
print("\nActive Ports:")
print(switch.active_ports)

```

Sample Output



Result

The StarSwitch class successfully simulated host connections, disconnections, and Ethernet broadcast operations.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done